

Relatório do Trabalho 01

**André Malmsteen Oliveira Amorim, Benjamim Isaac Ribeiro Lima,
Giovanna Bembom da Silva Bandeira, Mariana Ramos André Simões**

¹Instituto de Computação (IComp) – Universidade Federal do Amazonas (UFAM)
Av. Gen. Rodrigo Octávio 6200, Coroado I – 69080-900
Campus Universitário – Setor Norte – Manaus – AM – Brasil

{andre.mlmst, benjamim.lima}@icomp.ufam.edu.br,

{giovanna.bembom, mariana.simoes}@icomp.ufam.edu.br

1. Introdução

Este trabalho foi desenvolvido utilizando um computador pessoal com configurações adequadas para execução de experimentos computacionais e análise de desempenho de algoritmos. O dispositivo utilizado é um *notebook* Lenovo IdeaPad 3 15ITL6, equipado com processador Intel Core i5-1135G7 de 11ª geração, operando a uma frequência base de 2.40 GHz.

A máquina possui 8 GB de memória RAM com velocidade de 3200 MT/s, sendo aproximadamente 7,79 GB utilizáveis pelo sistema. O armazenamento total disponível é de 238 GB, dos quais cerca de 111 GB encontram-se em uso no momento da realização dos experimentos. Em relação ao processamento gráfico, o dispositivo conta com uma GPU integrada Intel Iris Xe Graphics.

O sistema operacional utilizado é o *Windows 11 Home Single Language*, versão 25H2, com arquitetura de 64 bits, garantindo compatibilidade com aplicações modernas e suporte a execução eficiente dos programas desenvolvidos.

Essas especificações fornecem uma base computacional suficiente para a implementação, execução e análise de algoritmos, permitindo a coleta de métricas de desempenho de forma confiável ao longo dos experimentos realizados neste trabalho.

Os códigos implementados estão disponíveis no GitHub, podendo ser acessado pelo seguinte *link*: Repositório Trabalho1_AEDII (https://github.com/giovannabembom12/Trabalho1_AEDII).

2. Busca sequencial com vetor desordenado VS Busca binária com vetor ordenado

Este programa implementa a comparação entre a busca sequencial em vetor desordenado e a busca binária em vetor ordenado para um vetor inicial de 1 milhão de valores inteiros. A execução ocorre em 30 repetições, sendo que nas primeiras 15 o valor buscado é garantidamente encontrado, já que sorteia-se uma posição do vetor, e nas 15 restantes gera-se um valor aleatório que provavelmente não está presente.

O programa principal `q1.c` (assim como todos os outros dois – `q2.c` e `q3.c`) utiliza as bibliotecas auxiliares `vetor.h`, que implementada como um Tipo Abstrato de Dados – TAD, e `estatisticas.h`, que possui funções matemáticas, como cálculo de tempo, média e desvio padrão.

A medição de tempo é feita com a função `tempoAtual()`, que é responsável por obter o tempo atual com alta resolução utilizando a estrutura `timespec`, que armazena o tempo em segundos (`tv_sec`) e nanossegundos (`tv_nsec`). A chamada `clock_gettime(CLOCK_MONOTONIC, &ts)` recupera o tempo decorrido de um relógio monotônico, que não sofre alterações por ajustes no relógio do sistema, garantindo maior precisão para medições de desempenho. Por fim, a função retorna o tempo total em segundos ao somar os segundos com a conversão dos nanossegundos para segundos (dividindo por 10^9), resultando em um valor do tipo `double` adequado para cálculos de intervalo de tempo.

Os resultados obtidos estão representados na Tabela 1.

Tabela 1: Tempos de 30 execuções de buscas sequenciais e binárias em um vetor de 1.000.000 elementos (em segundos)

Execução	Sequencial (s)	Binária (s)
01	0,000003	0,000002
02	0,000002	0,000000
03	0,000008	0,000001
04	0,000007	0,000001
05	0,000001	0,000000
06	0,000001	0,000000
07	0,000004	0,000000
08	0,000009	0,000000
09	0,000004	0,000001
10	0,000008	0,000000
11	0,000000	0,000000
12	0,000007	0,000001
13	0,000006	0,000000
14	0,000004	0,000000
15	0,000004	0,000001
16	0,000362	0,000001
17	0,000345	0,000000
18	0,000249	0,000000
19	0,000241	0,000000
20	0,000240	0,000000
21	0,000240	0,000000
22	0,000240	0,000000
23	0,000246	0,000000
24	0,000241	0,000000
25	0,000247	0,000000
26	0,000245	0,000000
27	0,000244	0,000000
28	0,000240	0,000000
29	0,000239	0,000000

Continua na próxima página

Tabela 1 – continuação

Execução	Sequencial (s)	Binária (s)
30	0,000240	0,000000
Média	0,000131	0,000000
Desvio Padrão	0,000131	0,000000

A Tabela 1 mostra que a busca sequencial apresentou média de 0,000131 segundos e desvio padrão de 0,000131 segundos, com valores muito baixos nas primeiras 15 execuções (entre 0 e 0,000009 segundos) e um patamar entre 0,000239 e 0,000362 segundos nas demais. Já a busca binária teve todos os tempos muito próximos a 0,000000 segundos (abaixo da resolução da medição), resultando em média e desvio nulos e evidenciando sua extrema eficiência para listas ordenadas.

A Figura 1 ilustra os tempos da busca sequencial (em azul) e da busca binária (em vermelho) para cada uma das 30 execuções. Nela, é fácil perceber que, enquanto a busca binária permanece inalterada no eixo zero, a busca sequencial apresenta dois patamares distintos: um primeiro grupo com tempos muito baixos (próximos de zero) e um segundo grupo com tempos entre 0,00024 segundos e 0,00036 segundos.

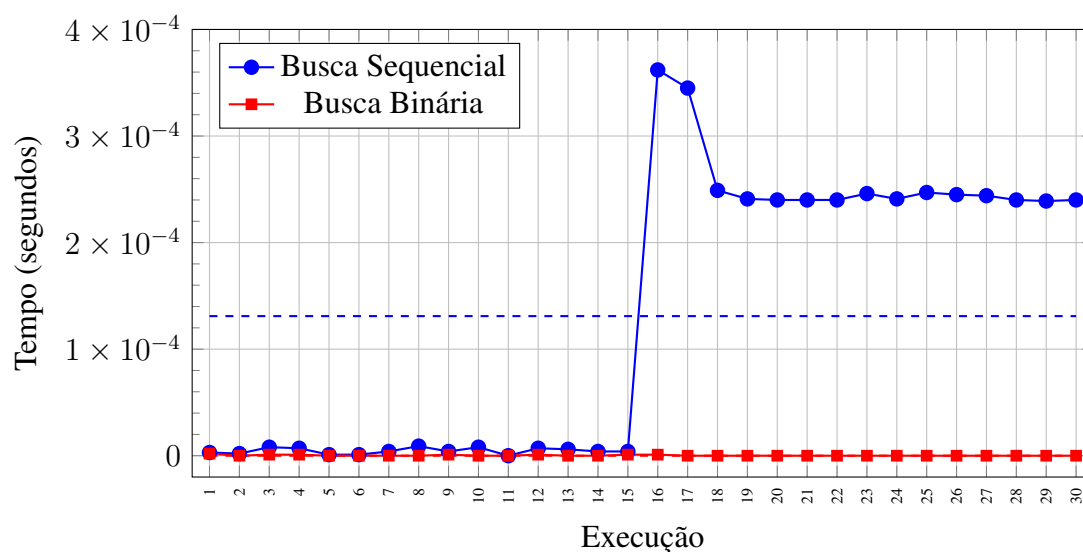


Figura 1. Tempos de 30 execuções de buscas sequenciais e binárias.

2.1. Aumentando o tamanho do vetor: 100.000.000 elementos

Com 1.000.000 elementos a busca binária tendia a 0, logo aumentou-se o número de elementos para 100.000.000 a fim de observar diferenças significativas nos tempos de execução. O algoritmo, no entanto, manteve as tendências anteriores, apesar de possuir mais elementos e tempos maiores.

Os tempos individuais para esse vetor reforçam o comportamento bimodal na busca sequencial: as primeiras 15 execuções são extremamente rápidas (média de aproximadamente 0,000005 segundos), enquanto as 15 execuções seguintes apresentam tem-

pos entre 0,030 segundos e 0,055 segundos, condizentes com a necessidade de percorrer grande parte do vetor. A média geral da sequencial é 0,016266 segundos com desvio padrão elevado, refletindo essa disparidade. Por outro lado, a busca binária mantém tempos praticamente nulos (média de 0,000001 segundos e desvio padrão de 0,000002 segundos), evidenciando sua complexidade logarítmica $O(\log n)$ e a enorme vantagem sobre a busca linear quando os dados estão ordenados.

Os tempos de execução desse segundo experimento foram registrados e são apresentados na Tabela 2, juntamente a média e desvio padrão de cada um dos tipos de busca.

Tabela 2: Tempos de 30 execuções de buscas sequenciais e binárias em vetor de 100.000.000 elementos (em segundos)

Execução	Sequencial (s)	Binária (s)
01	0,000004	0,000001
02	0,000006	0,000001
03	0,000003	0,000000
04	0,000011	0,000001
05	0,000001	0,000000
06	0,000006	0,000001
07	0,000002	0,000000
08	0,000008	0,000000
09	0,000004	0,000000
10	0,000005	0,000000
11	0,000001	0,000000
12	0,000006	0,000000
13	0,000000	0,000000
14	0,000004	0,000001
15	0,000007	0,000001
16	0,055309	0,000002
17	0,030543	0,000001
18	0,030676	0,000002
19	0,030589	0,000002
20	0,031517	0,000002
21	0,030574	0,000002
22	0,030461	0,000001
23	0,030491	0,000001
24	0,031603	0,000009
25	0,031463	0,000001
26	0,031025	0,000002
27	0,030912	0,000001
28	0,031612	0,000002
29	0,030749	0,000003
30	0,030394	0,000001
Média	0,016266	0,000001

Continua na próxima página

Tabela 2 – continuação

Execução	Sequencial (s)	Binária (s)
Desvio Padrão	0,017112	0,000002

A Figura 2 ilustra os comportamentos observados na Tabela 2: a busca sequencial (representada pela cor azul) apresenta um primeiro patamar próximo de zero e um segundo patamar na faixa dos milissegundos, enquanto a binária (de cor vermelha) permanece inalterada sobre o eixo horizontal, representando sua rapidez.

É importante ressaltar, que, por questões comparativas, implementou-se a versão do código que utilizava um vetor com 1.000.000.000 elementos, porém, a partir desse momento, tornou-se inviável a execução do código na máquina utilizada.

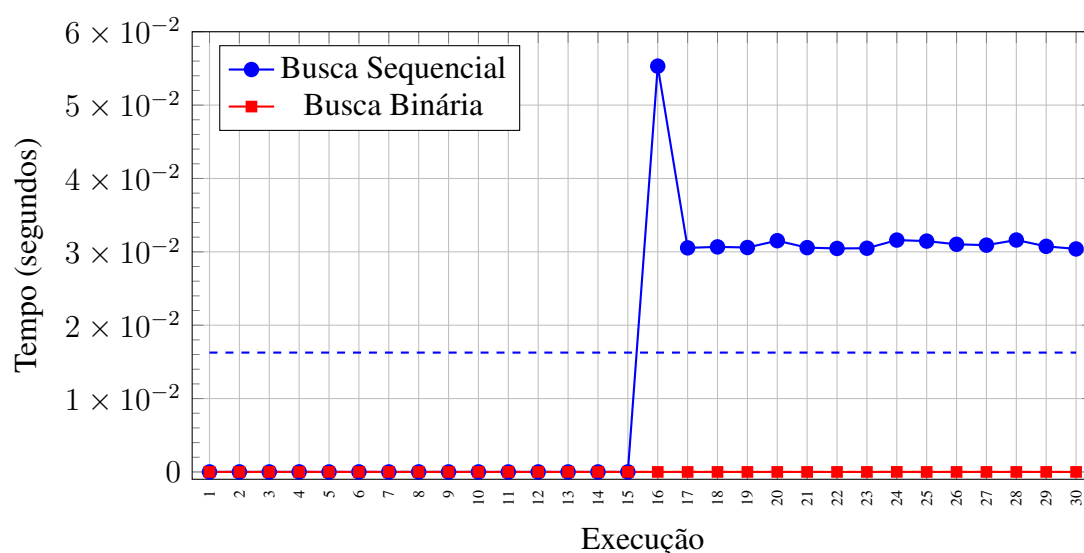


Figura 2. Tempos de 30 execuções de buscas sequenciais e binárias em vetor de 100.000.000 elementos (em segundos).

2.2. Comparação de tempos médios de 100.000 até 1.000.000

Além das medições anteriores, os tempos médios de execução para os algoritmos de busca sequencial e binária também foram calculados variando o número de elementos de 100.000 a 1.000.000. Os resultados obtidos são apresentados na Tabela 3. Observa-se que o tempo da busca sequencial cresce conforme o tamanho da entrada aumenta, enquanto a busca binária apresenta tempos tão pequenos que não puderam ser registrados com a precisão utilizada (0.000000 segundos), evidenciando mais uma vez sua alta eficiência para dados ordenados.

Tabela 3: Tempos médios de busca sequencial e binária variando de 100.000 a 1.000.000 elementos (em segundos)

Número de elementos	Sequencial (s)	Binária (s)
100.000	0.000014	0.000000
200.000	0.000032	0.000000
300.000	0.000039	0.000000
400.000	0.000043	0.000000
500.000	0.000066	0.000000
600.000	0.000078	0.000000
700.000	0.000092	0.000000
800.000	0.000104	0.000000
900.000	0.000120	0.000000
1.000.000	0.000166	0.000000

O comportamento observado está de acordo com a complexidade teórica dos algoritmos. A busca sequencial percorre cada elemento até encontrar o alvo (ou até o final da lista), resultando em complexidade $O(n)$. Por isso, seu tempo de execução aumenta linearmente com o número de elementos. Já a busca binária apresenta complexidade $O(\log n)$. Esse crescimento logarítmico é tão eficiente que o tempo de execução permanece abaixo da resolução da medição, justificando os valores zero registrados.

A Figura 3 ilustra a evolução dos tempos médios para os dois algoritmos. Enquanto a busca sequencial mostra uma tendência linear clara, a busca binária permanece praticamente nula na escala do gráfico.

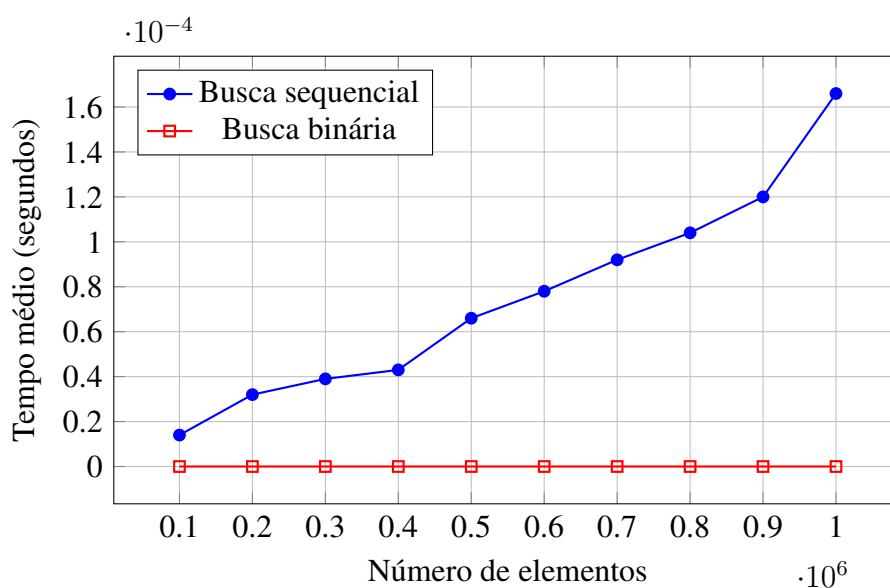


Figura 3. Comparação de tempos médios de busca sequencial e binária variando de 100.000 a 1.000.000 elementos (em segundos)

3. Busca sequencial em vetor desordenado VS Busca sequencial em lista encadeada

O programa compara o desempenho da busca sequencial em duas estruturas de dados: um vetor desordenado e uma lista encadeada. Ambos armazenam 1 milhão de inteiros gerados aleatoriamente e não ordenados. São realizadas 30 execuções para cada estrutura, utilizando os mesmos valores procurados em cada iteração, de modo a permitir a comparação direta dos tempos.

A Tabela 4 apresenta os valores obtidos em cada busca.

Tabela 4: Tempos de 30 execuções de busca sequencial em vetor desordenado e lista encadeada com 1.000.000 elementos (em segundos).

Execução	Vetor (s)	Lista (s)
01	0.000210	0.004138
02	0.000093	0.001875
03	0.000281	0.006957
04	0.000446	0.009560
05	0.000327	0.005842
06	0.000053	0.001084
07	0.000380	0.007751
08	0.000353	0.008247
09	0.000235	0.004539
10	0.000062	0.001187
11	0.000006	0.000149
12	0.000070	0.001234
13	0.000274	0.005102
14	0.000011	0.000189
15	0.000031	0.000599
16	0.000399	0.009138
17	0.000473	0.008897
18	0.000495	0.008693
19	0.000395	0.011433
20	0.000921	0.008786
21	0.000871	0.018491
22	0.000852	0.015067
23	0.000500	0.009413
24	0.000659	0.009771
25	0.000406	0.012644
26	0.000669	0.009873
27	0.000397	0.007833
28	0.000386	0.008235
29	0.000365	0.007794
30	0.000399	0.007910

Continua na próxima página

Tabela 4 – continuação		
Execução	Vetor (s)	Lista (s)
Média	0.000367	0.007081

Conforme apresentado na Tabela 4, os tempos de busca sequencial no vetor são significativamente menores do que na lista encadeada para todas as 30 execuções. Enquanto o vetor apresentou tempos individuais variando entre 0,000006 s e 0,000921 s (média de 0,000367 s), a lista obteve tempos entre 0,000149 s e 0,018491 s (média de 0,007081 s).

Essa diferença de aproximadamente uma ordem de magnitude (cerca de 19 vezes mais lento para a lista) evidencia o impacto da localidade de referência: o vetor armazena os dados em posições contíguas na memória, permitindo acesso rápido e eficiente ao percorrer os elementos. Já a lista encadeada, por armazenar os nós em posições potencialmente esparsas da memória, sofre com sucessivas falhas de cache (*cache misses*), o que degrada consideravelmente o desempenho da busca sequencial.

A Figura 4 ilustra melhor essa diferença.

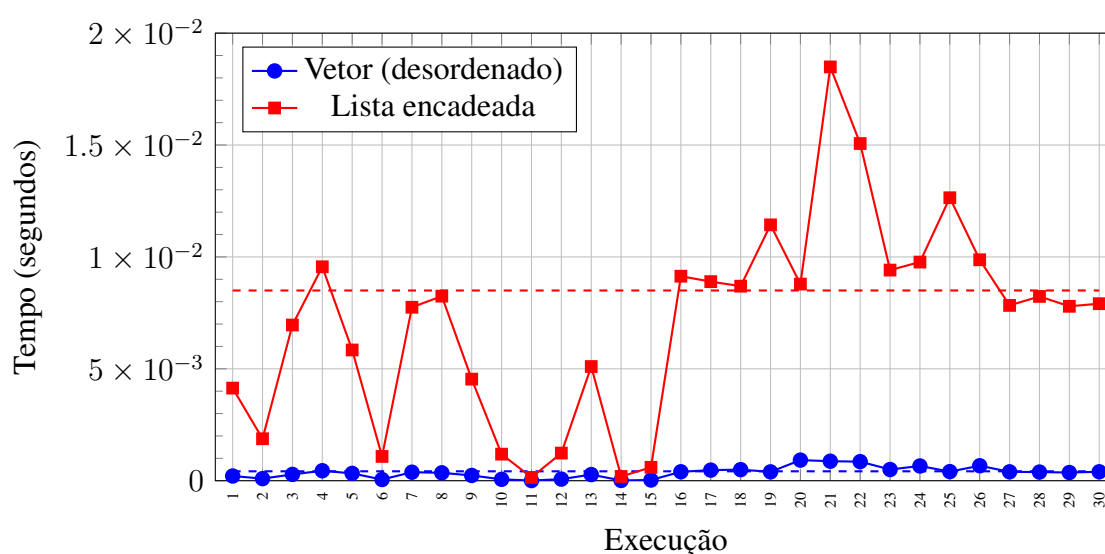


Figura 4. Tempos de 30 execuções de busca sequencial em vetor desordenado e lista encadeada com 1.000.000 elementos.

3.1. Aumentando o tamanho do vetor: 10.000.000 elementos

Assim como no Tópico 2 (que tratava a comparação entre a Busca Sequencial e a Busca Binária em vetores), neste tópico foi necessário aumentar a quantidade de elementos do vetor e da lista, para fins de comparação.

De maneira equivalente ao experimento com 1.000.000 elementos, os resultados desse experimento apresentam uma diferença significativa entre os tempos de busca em um vetor e em uma lista encadeada, ambos contendo 10 milhões de elementos. Novamente, foram realizadas 30 execuções de busca sequencial para cada estrutura. O tempo

médio para o vetor foi de 0,002763 segundos, enquanto para a lista encadeada foi de 0,052323 segundos, ou seja, a lista foi aproximadamente 19 vezes mais lenta.

Essa disparidade decorre pelo mesmo motivo do tópico anterior.

Na Tabela 5, é possível observar o tempo exato de cada uma das execuções, além das médias correspondentes.

Tabela 5: Tempos de 30 execuções de busca sequencial em vetor desordenado e lista encadeada com 10.000.000 elementos (em segundos).

Execução	Vetor (s)	Lista (s)
01	0.002161	0.049579
02	0.000052	0.001089
03	0.001559	0.030782
04	0.002768	0.064441
05	0.000619	0.006497
06	0.000570	0.013884
07	0.000433	0.008209
08	0.002814	0.069193
09	0.000239	0.005231
10	0.000853	0.022955
11	0.007120	0.113914
12	0.003930	0.060663
13	0.004660	0.039706
14	0.003298	0.062831
15	0.001717	0.033633
16	0.003533	0.070271
17	0.003059	0.065821
18	0.002959	0.064210
19	0.002957	0.065895
20	0.004737	0.062562
21	0.002967	0.062752
22	0.003590	0.069000
23	0.002961	0.065626
24	0.002970	0.074995
25	0.003438	0.068257
26	0.003236	0.063397
27	0.003112	0.062803
28	0.002967	0.062257
29	0.002964	0.063579
30	0.004641	0.065670
Média	0.002763	0.052323

Conclui-se, portanto, que, mesmo nos melhores casos para a lista, como por exem-

plo a Busca 02 (0,001089 segundos) e a Busca 07 (0,008209 segundos), o vetor manteve ordens de grandeza significativamente inferiores.

A Figura 5, representada a seguir, ajuda na visualização dessa diferença:

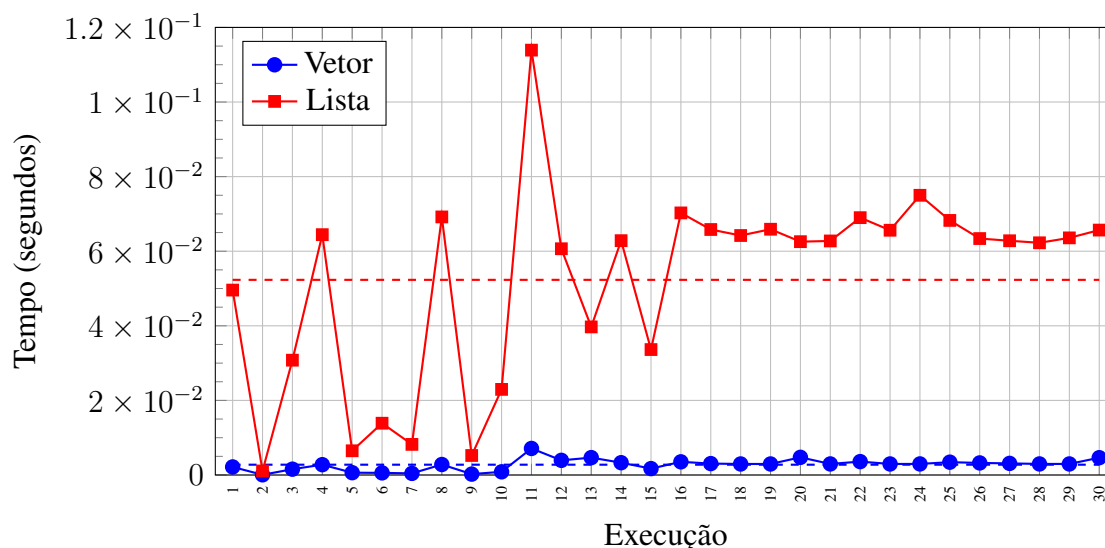


Figura 5. Tempos de 30 execuções de busca sequencial em vetor desordenado e lista encadeada com 10.000.000 elementos.

3.2. Comparação de tempos médios de 100.000 até 1.000.000

Para essa comparação, os testes variaram o número de elementos de 100.000 a 1.000.000, medindo ainda o tempo médio de busca sequencial em um vetor e em uma lista encadeada. Porém, dessa vez, busca-se comparar o crescimento do tempo de execução em relação a quantidade de elementos no vetor e na lista.

Os resultados, apresentados na Tabela 6, comprovam que o vetor é consistentemente mais rápido que a lista. Para 100.000 elementos, o vetor levou 0,000036 segundos contra 0,000532 segundos da lista, o que a torna cerca de 15 vezes mais lenta. Conforme o tamanho cresce, a diferença se acentua: em 500.000 elementos, vetor 0,000161 s e lista 0,003091 s (19 vezes); em 1.000.000 elementos, vetor 0,000367 s e lista 0,007081 s (cerca de 19,3 vezes). A lista apresenta crescimento aproximadamente linear, enquanto o vetor também cresce linearmente, mas com inclinação muito menor devido à melhor localidade de memória e ausência de ponteiros.

Tabela 6: Tempos médios de busca sequencial em vetor e lista variando de 100.000 a 1.000.000 elementos (em segundos).

Número de elementos	Vetor (s)	Lista (s)
100.000	0.000036	0.000532
200.000	0.000067	0.001043

Continua na próxima página

Tabela 6 – continuação

Número de elementos	Vetor (s)	Lista (s)
300.000	0.000103	0.001594
400.000	0.000125	0.002200
500.000	0.000161	0.003091
600.000	0.000204	0.003393
700.000	0.000219	0.003719
800.000	0.000261	0.004392
900.000	0.000323	0.005134
1.000.000	0.000367	0.007081

A Figura 6 ilustra essa diferença: enquanto o vetor mantém uma subida suave, a lista apresenta inclinação mais acentuada, refletindo seu maior custo por elemento.

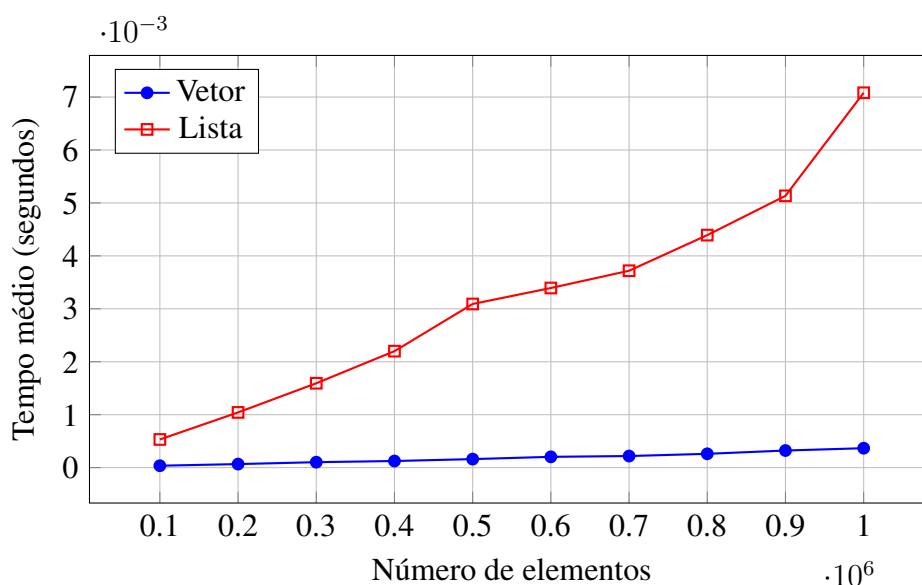


Figura 6. Tempos médios de busca sequencial em vetor e lista variando de 100.000 a 1.000.000 elementos (em segundos).

4. Algoritmos de ordenação em vetor

O programa implementa a comparação de desempenho entre cinco algoritmos de ordenação: *Bubble Sort*, *Insertion Sort*, *Selection Sort*, *Quick Sort* e *Merge Sort*. O vetor de trabalho possui 100.000 elementos inteiros preenchidos com valores aleatórios não ordenados.

O *Selection Sort* obteve média de 28,07 segundos. Apesar da complexidade $O(n^2)$, sua vantagem sobre o *Bubble Sort* reside no menor número de trocas, $O(n)$, embora a instabilidade e o custo de comparações limitem seu uso. Já o *Insertion Sort* destacou-se como o mais eficiente entre os algoritmos quadráticos, com média de 3,72s. Sua característica adaptativa e eficiente localidade de referência permitiram um desempenho até dez vezes superior ao *Selection Sort* neste cenário. Os algoritmos de divisão e

conquista, *Merge Sort* e *Quick Sort*, demonstraram superioridade absoluta com médias de 0,0337 segundos e 0,0167 segundos, respectivamente. O *Merge Sort* garante $O(n \log n)$ em todos os casos ao custo de memória extra $O(n)$. O *Quick Sort*, embora vulnerável ao pior caso $O(n^2)$, confirmou sua eficiência prática superior devido à constante multiplicativa reduzida e melhor uso do sistema de *cache*, superando o *Merge Sort* em todas as execuções. A Tabela 7, representada a seguir, detalha os tempos individuais.

Tabela 7: Tempos de execução (em segundos) para cinco algoritmos de ordenação.

Execução	Bubble	Selection	Insertion	Merge	Quick
1	29,072256	12,730504	1,711178	0,014576	0,007203
2	29,381455	12,440657	1,773046	0,014141	0,007049
3	28,256720	12,322568	1,635676	0,013753	0,007070
4	27,953417	12,364062	1,614079	0,013717	0,006975
5	379,352404	33,645536	5,374157	0,045729	0,022430
6	86,401038	39,746884	4,814851	0,045876	0,024462
7	91,551702	40,235609	5,185905	0,046308	0,023443
8	89,665733	38,975967	4,946510	0,046491	0,027161
9	91,851587	40,356565	5,383940	0,046008	0,024902
10	89,368751	37,924019	4,767189	0,050499	0,016599
Média	94,285506	28,074237	3,720653	0,033710	0,016729
Desvio Padrão	104,673708	13,567421	1,765683	0,016979	0,008730

Abaixo, a Figura 7 ilustra a dispersão dos dados em escala logarítmica. Nela, o *outlier* é visível e justifica o desvio padrão e a média elevados.

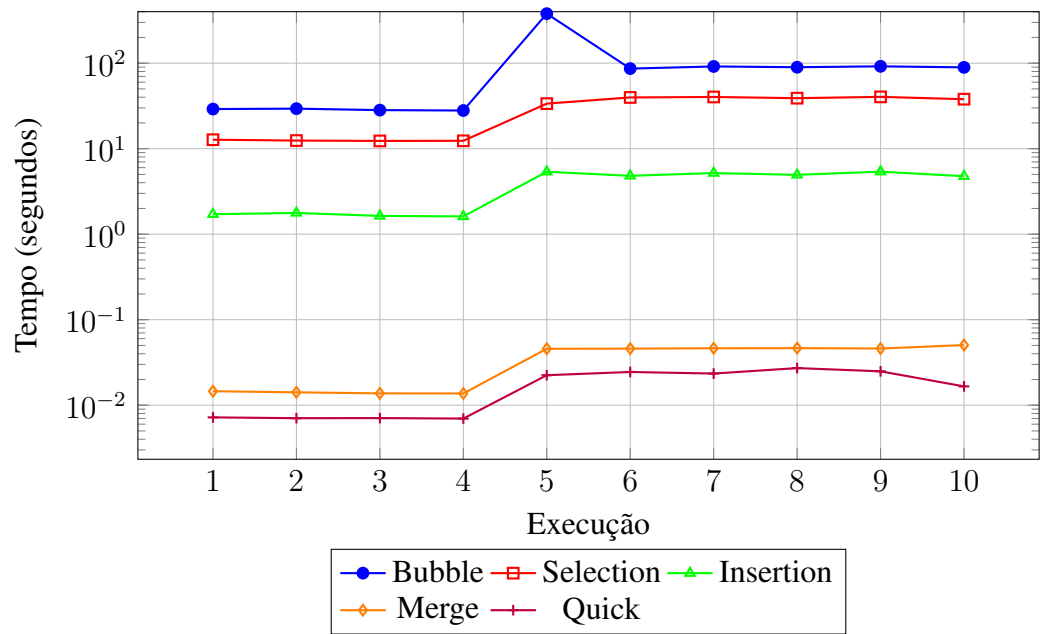


Figura 7. Tempos individuais das 10 execuções (escala logarítmica).

A Figura 8 sintetiza as médias e desvios padrão, evidenciando a estabilidade dos algoritmos $O(n \log n)$ frente à alta variabilidade dos modelos quadráticos sob carga elevada.

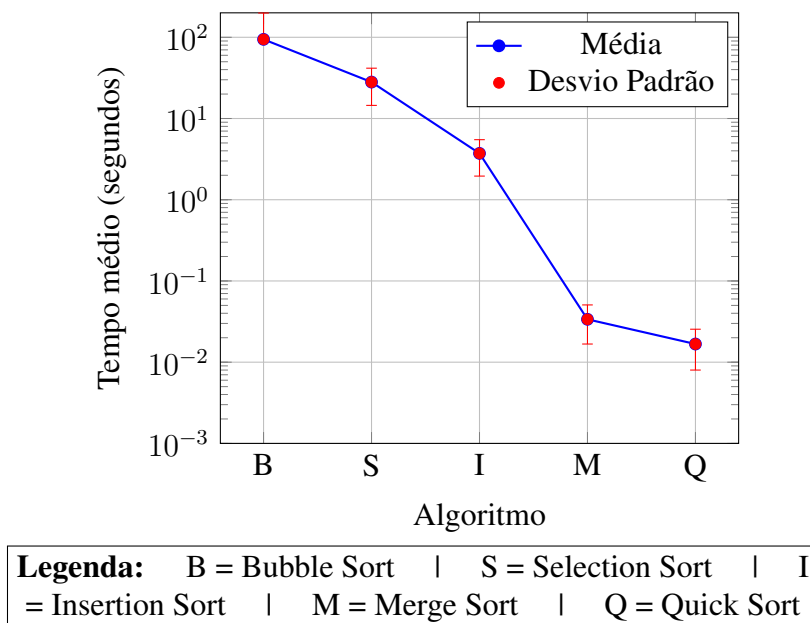


Figura 8. Médias com barras de desvio padrão (escala log).

4.1. Comparação de tempos médios de 10.000 até 100.000

Foram medidos os tempos médios de execução (em segundos) dos algoritmos *Bubble Sort*, *Selection Sort*, *Insertion Sort*, *Merge Sort* e *Quick Sort* para vetores de tamanhos variando de 10.000 a 100.000 elementos. Os resultados são apresentados na Tabela 8. Observa-se que, para cada algoritmo, o tempo cresce com o aumento do número de elementos, porém com taxas muito distintas. Os algoritmos quadráticos (*Bubble*, *Selection*, *Insertion*) apresentam crescimento acelerado, enquanto os de complexidade $O(n \log n)$ (*Merge* e *Quick*) mantêm tempos muito baixos mesmo para 100.000 elementos.

Dessa forma, a Tabela 8 e a Figura 9 apresentam a escalabilidade dos algoritmos com N variando de 10.000 a 100.000. A inclinação das curvas em escala log-log ratifica as complexidades teóricas: aproximadamente 2 para os quadráticos e próxima de 1 para os log-lineares. O salto anômalo do *Bubble Sort* no limite de 100.000 elementos sugere gargalos de *hardware* ou paginação de memória, reforçando a obsolescência de algoritmos ineficientes para processamento de grandes conjuntos de dados.

Tabela 8: Tempos médios (em segundos) por tamanho do vetor

N	Bubble	Selection	Insertion	Merge	Quick
10.000	0,164237	0,096308	0,012723	0,001012	0,000486
20.000	0,781157	0,390155	0,051355	0,002118	0,001055

Continua na próxima página

Tabela 8 – continuação

N	Bubble	Selection	Insertion	Merge	Quick
30.000	1,922770	0,899126	0,122807	0,003250	0,001620
40.000	3,341986	1,543021	0,207407	0,004436	0,002202
50.000	5,331607	2,429611	0,320658	0,006008	0,002870
60.000	7,712831	3,467145	0,456409	0,006649	0,003617
70.000	10,516218	4,718900	0,622159	0,007648	0,003979
80.000	13,875984	6,171965	0,816847	0,008954	0,004582
90.000	17,574791	7,815458	1,026363	0,009982	0,005187
100.000	94,285506	28,074237	3,720653	0,033710	0,016729

A Figura 9 utiliza escalas logarítmicas em ambos os eixos para melhor visualização das diferentes taxas de crescimento. Os algoritmos quadráticos apresentam inclinação aproximadamente igual a 2, confirmando a complexidade $O(n^2)$. Já o *Merge Sort* e o *Quick Sort* mostram inclinação próxima de 1, compatível com $O(n \log n)$, sendo que o *Quick Sort* mantém-se abaixo do *Merge* em todo o intervalo. O salto do *Bubble Sort* em 100.000 elementos é anômalo (aumento muito superior ao esperado), podendo indicar alguma interferência externa (como *cache misses* ou troca para memória virtual). Apesar disso, a tendência geral dos algoritmos quadráticos é clara: tornam-se inviáveis para tamanhos acima de dezenas de milhares de elementos, enquanto os algoritmos eficientes executam a ordenação em frações de segundo mesmo para 100.000 entradas.

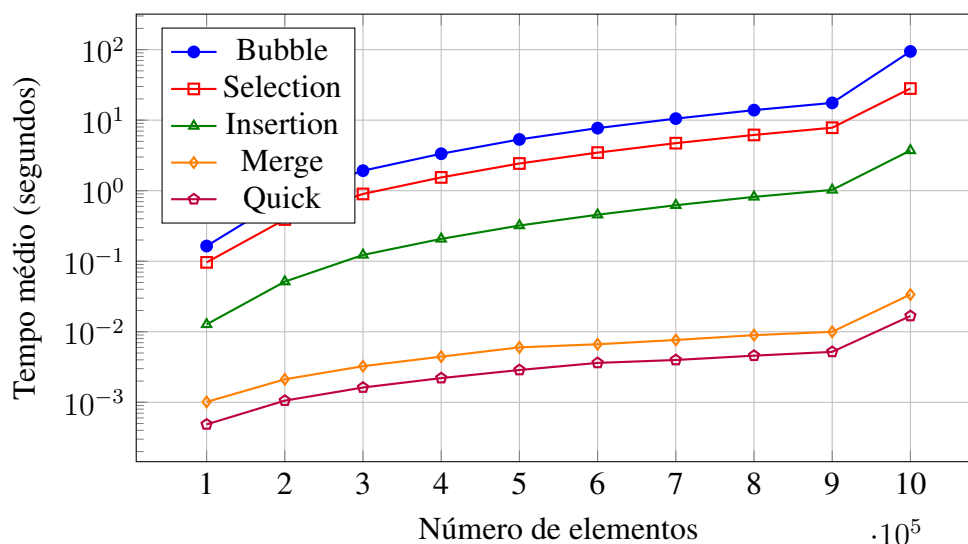


Figura 9. Comparação dos tempos médios de ordenação por algoritmo (escala logarítmica no eixo y).

5. Conclusão

Este trabalho apresentou a implementação e análise experimental de diferentes estruturas de dados e algoritmos de busca e ordenação, permitindo observar na prática o impacto das complexidades teóricas no desempenho computacional.

No primeiro experimento, a comparação entre busca sequencial e busca binária evidenciou de forma clara a superioridade da complexidade $O(\log n)$ frente à $O(n)$. Mesmo para vetores de até 100 milhões de elementos, a busca binária manteve tempos de execução abaixo da resolução do instrumento de medição, enquanto a busca sequencial apresentou crescimento linear perceptível já a partir de 1 milhão de elementos — atingindo médias na casa dos centésimos de segundo para o vetor maior.

No segundo experimento, a comparação entre busca sequencial em vetor desordenado e em lista encadeada revelou a importância da localidade de referência na memória. Embora ambas as estruturas apresentem complexidade $O(n)$, o vetor mostrou-se consistentemente cerca de 19 vezes mais rápido do que a lista encadeada, independentemente do tamanho da entrada testado. Esse resultado demonstra que a análise assintótica, por si só, não é suficiente para prever o desempenho real de um algoritmo: fatores de arquitetura, como a taxa de *cache misses*, exercem influência decisiva na prática.

No terceiro experimento, a comparação entre os cinco algoritmos de ordenação confirmou a superioridade dos algoritmos de divisão e conquista. O *Quick Sort* e o *Merge Sort*, ambos $O(n \log n)$, superaram amplamente os algoritmos quadráticos ($O(n^2)$) — *Bubble Sort*, *Selection Sort* e *Insertion Sort* —, que se tornaram inviáveis para entradas acima de dezenas de milhares de elementos. O *Quick Sort* destacou-se como o mais eficiente na prática, devido à sua melhor constante multiplicativa e ao uso mais eficiente do *cache*, confirmando seu desempenho superior mesmo em relação ao *Merge Sort*, apesar da vulnerabilidade teórica ao pior caso.

Em conjunto, os experimentos reforçam que a escolha adequada de estruturas de dados e algoritmos é determinante para a eficiência computacional, e que o comportamento empírico, embora geralmente coerente com a teoria, pode ser significativamente influenciado por fatores de *hardware* e pelo ambiente de execução.